

Semantic Emotion Recognition

Zeping Ruan, Yiyang Tan, Ge Gao
{ruanzp2023,tanyy2023,gaoge2023}@shanghaitech.edu.cn

Abstract

This project investigates Semantic Emotion Recognition on game comment data using an enhanced Naive Bayes classifier and a Hidden Markov Model. We report the prediction accuracy of each model and analyze their effectiveness and limitations in capturing emotional semantics.

1 Introduction

Identifying the mood of user comments has become a core task in modern online environments, enabling applications such as content filtering, trend analysis, and hotspot tracking. This project applies two classic machine learning approaches to perform semantic emotion recognition on game-related comments. Our primary contributions are as follows:

- We propose a complete pipeline for identifying positive and negative sentiments specifically tailored to game comment data.
- We implement a systematic preprocessing framework to clean and normalize raw user comments.
- We develop and evaluate two models: an Enhanced Naive Bayes classifier and a Hidden Markov Model (HMM).
- We provide a comparative analysis of the predictive accuracies of the two methods, along with a detailed discussion of their respective strengths and limitations.

The steam comment dataset we used can be found at Kaggle: [Steam Games, Reviews, and Rankings](#).

2 Methodology

2.1 Data Preprocessing

To ensure consistency and reduce noise in the input data, we applied a standard text preprocessing pipeline. Each comment was first lowercased and stripped of HTML tags and URLs using regular expressions. Non-alphanumeric characters were removed, except for spaces to preserve word boundaries. Finally, multiple spaces were collapsed and leading/trailing whitespace was trimmed. This process yields clean textual inputs suitable for tokenization and feature extraction.

2.2 Naive Bayes

2.2.1 Traditional Naive Bayes

The traditional Naive Bayes is a probabilistic model based on Bayes' theorem with the assumption of conditional independence between features given the class label. In this project, we employ the Naive Bayes classifier to model the probability that a given sentence (comment) expresses either a positive or a negative sentiment.

Formally, given a sentence $d = \{w_1, w_2, \dots, w_n\}$ consisting of words w_i from the vocabulary, and a class label $c \in \{Positive, Negative\}$, the classifier assigns a label according to the maximum a posteriori (MAP) estimate:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} P(c | d) = \arg \max_{c \in \mathcal{C}} P(c) \prod_{i=1}^n P(w_i | c)$$

Here, $P(c)$ is the prior probability of class c , and $P(w_i | c)$ is the likelihood of word w_i appearing in documents of class c . These probabilities are typically estimated using maximum likelihood from training data:

$$P(w | c) = \frac{N_{wc}}{N_c}$$

where N_{wc} is the number of occurrences of word w in documents of class c , N_c is the total number of word occurrences in class c .

2.2.2 Enhancement on traditional Naive Bayes

Following Narayanan et al. (2011)[1], we augment the standard Multinomial Naive Bayes with four key techniques—additive smoothing, negation handling, n -gram features, and mutual-information-based feature selection—to obtain both high accuracy and linear training/testing time.

Additive Laplace smoothing: To avoid zero-probability issues for unseen words, we apply add-1 smoothing when estimating class-conditional likelihoods:

$$P(w | c) = \frac{\text{count}(w, c) + 1}{\sum_{w'} (\text{count}(w', c) + 1)} \quad (1)$$

where $\text{count}(w, c)$ is the number of documents in class c containing word w .

Negation handling: We follow the “bootstrapped negation” scheme: in each document, upon encountering “not” or “n’t” we flip a Boolean negation state until the next punctuation. All subsequent tokens t are tagged as “not_ t ” in both training and testing. Moreover, during training we increment the count of “not_ t ” in the opposite class of t ’s normal occurrences, ensuring sufficient statistics for negated forms.

n -gram augmentation: In addition to unigrams, we extract and count the most frequent bigrams and trigrams from the corpus. Each n -gram is treated as an atomic feature in the same Multinomial NB framework, allowing the model to capture local phrasal sentiment (e.g. “very good”, “not worth”).
Feature selection via mutual information: To reduce noise and dimensionality, we compute the mutual information between each feature x and the class label $Y \in \{\text{Pos}, \text{Neg}\}$:

$$I(x; Y) = \sum_{x' \in \{0,1\}} \sum_{y \in \{\text{Pos}, \text{Neg}\}} P(x' = x, Y = y) \log \frac{P(x' = x, Y = y)}{P(x' = x) P(Y = y)}, \quad (2)$$

where $x' = 1$ indicates presence of the feature. We rank all unigrams and n -grams by $I(x; Y)$ and retain only the top K features (we found $K = 32,000$ to be optimal on a 1 000-document validation set).

Final decision rule: Under these enhancements, for a new document $d = \{w_1, \dots, w_n\}$ the posterior estimate becomes

$$\hat{c} = \arg \max_{c \in \{\text{Pos}, \text{Neg}\}} \left[\log P(c) + \sum_{i=1}^n \log P(w_i | c) \right],$$

where each w_i may be a unigram, an n -gram, or a negation-tagged token, and all $P(\cdot | c)$ have been smoothed and prefiltered by mutual information.

2.3 Hidden Markov Model

The Hidden Markov Model (HMM)[2] is a generative probabilistic model that represents a sequence of observable events as emissions from a sequence of hidden states. It is particularly well-suited for modeling sequential data with the assumption that process follows the Markov property.

Formally, an HMM is defined by:

- A set of hidden states $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$;

- An observation vocabulary $\mathcal{O} = \{o_1, o_2, \dots, o_M\}$;
- An initial state distribution $\pi = \{\pi_i\}$ where $\pi_i = P(s_1 = s_i)$;
- A state transition probability matrix $A = \{a_{ij}\}$ where $a_{ij} = P(s_{t+1} = s_j \mid s_t = s_i)$;
- An emission probability matrix $B = \{b_j(o)\}$ where $b_j(o) = P(o_t = o \mid s_t = s_j)$.

Given an observed sequence of tokens $O = (o_1, o_2, \dots, o_T)$ (sentence), the goal is to infer the most probable sequence of hidden emotional states $S = (s_1, s_2, \dots, s_T)$ that generated the observations. This is commonly achieved using the Viterbi algorithm, which maximizes the joint probability:

$$\hat{S} = \arg \max_S P(S, O \mid \lambda)$$

where $\lambda = (\pi, A, B)$ denotes the model parameters.

3 Implementations & Results

3.1 Dataset

We selected training data from entries ranked between 300,000 and 800,000 in the dataset, filtered to include only reviews where the playtime exceeds 60 hours and the number of users who found the review helpful is above 15. This filtering strategy helps ensure higher-quality textual data, which in turn contributes to building a more effective classification model. Furthermore, we observed that the overall dataset exhibits an approximate 3:1 ratio between positive and negative reviews. To preserve this distribution, we constructed a training set that reflects a similar class imbalance.

We also constructed the test set by similarly selecting entries from the first 100,000 samples where the playtime exceeds 60 hours and the number of users who found the review helpful is greater than 15. This filtering resulted in a total of **5,374** positive reviews and **3,384** negative reviews.

3.2 Enhanced Naive Bayes

We implement the Enhanced Naive Bayes algorithm described in 2.2.2. After tuning our model, the final selected hyperparameters are shown in Figure 1.

Throughout the training process, we observed a notable increase in classification accuracy. The progression of training accuracy is shown in Figure 2.

Parameter	Value
Positive sentences selected	22562
Negative sentences selected	6602
Positive sentences used	10000
Negative sentences used	6602
Features selected	20000~60000
Step	200

Figure 1: Naive Bayes Training Settings

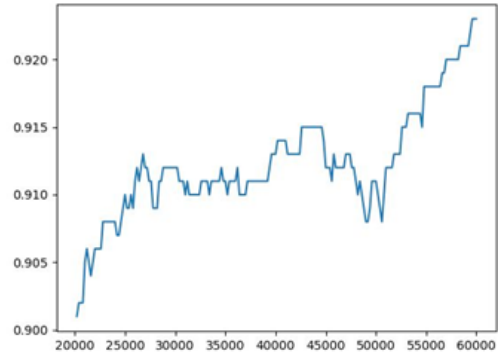


Figure 2: Model Accuracy

After training, we test the model on our test set, the result is shown in Table 1

Table 1: Performance of the Enhanced Naive Bayes on the Test Set

Metric	Value
True Positives (TP)	3551
False Positives (FP)	509
True Negatives (TN)	2875
False Negatives (FN)	1823
Precision	0.8746
Recall	0.6608
Accuracy (%)	73.37

3.3 Hidden Markov Model

We implement the discrete-observation HMM classifier as described in Section 2.3. The procedure is: **Vocabulary construction:** Retain the top 20,000 features by mutual information and add a special ‘<UNK>’ token for out-of-vocabulary words.

Sequence conversion: For each review, apply the `negate_sequence` function to handle negations, then map each token to its index in the vocabulary, producing an observation sequence and its length.

Model training: Split the training data into positive and negative subsets. Train two HMMs (one for positive, one for negative) using $N = 3$ hidden states and up to 20 EM iterations each.

Classification rule: For a new review, compute its log-likelihood under each HMM:

$$\ell_{\text{pos}} = \log P(\text{obs} \mid \text{HMM}_{\text{pos}}), \quad \ell_{\text{neg}} = \log P(\text{obs} \mid \text{HMM}_{\text{neg}}).$$

Predict “positive” if $\ell_{\text{pos}} > \ell_{\text{neg}}$, otherwise “negative.” During EM training, the log-likelihood for both models stabilizes after approximately 20 iterations. We evaluated the HMM classifier on the same test set, results are shown in Table 2.

Table 2: Performance of the HMM Classifier on the Test Set

Metric	Value
True Positives (TP)	4677
False Positives (FP)	1368
True Negatives (TN)	2016
False Negatives (FN)	697
Precision	0.7737
Recall	0.8703
Accuracy (%)	76.42

The HMM classifier now achieves an overall accuracy of 76.42%, with precision of 77.37% and recall of 87.03%. Compared to the Enhanced Naive Bayes model (73.37% accuracy, 87.46% precision, 66.08% recall), the HMM shows:

- Higher recall (87.03% vs. 66.08%), indicating it captures a larger fraction of positive reviews.
- Lower precision (77.37% vs. 87.46%), resulting in more false positives.
- Improved overall accuracy (76.42% vs. 73.37%), demonstrating the benefit of modeling sequence information.

This suggests that the HMM’s sequence-based approach better identifies positive sentiment at the cost of increased false positives. Its ability to capture temporal patterns—such as negation scope and sentiment shifts—provides a valuable complement to bag-of-words methods. Future work may explore hybrid models combining HMM outputs with Naive Bayes probabilities to strike a better balance between precision and recall.

3.4 Game Ranking

3.4.1 Game Data & Metrics

In our selected test set, we further counted the number of comments associated with each game. The games selected is shown in Table 3, we ultimately included only games with more than 100 comments for ranking purposes, since games with too few comments may introduce significant statistical bias. Although we have already filtered for high-quality reviews during data selection, the number of comments per game remains relatively limited in the ranking stage. To further improve the reliability of the ranking results, we aim to more precisely estimate the weight of each individual comment based on its associated metadata.

Specifically, we compute the normalized values of both `hours_played` and `helpful` for each review as follows:

$$\text{normalized_hours_played} = \frac{\text{hours_played} - h_{\min}}{h_{\max} - h_{\min}} \quad (3)$$

$$\text{normalized_helpful} = \frac{\text{helpful} - u_{\min}}{u_{\max} - u_{\min}} \quad (4)$$

Here, $h_{\min}$ and $h_{\max}$ denote the minimum and maximum values of playtime across all reviews, while $u_{\min}$ and $u_{\max}$ represent the corresponding bounds for the helpfulness score.

The final weight assigned to each comment is computed as a linear combination of the two normalized attributes:

$$\text{weighted_score} = 0.5 \cdot \text{normalized_hours_played} + 0.5 \cdot \text{normalized_helpful} \quad (5)$$

To calculate the overall sentiment score for each game, we iterate through all its associated comments. For every comment predicted as positive, we add its weight to the game score; for every comment predicted as negative, we subtract the weight. The final score is then normalized by the total number of comments for that game:

$$\text{game_score} = \frac{1}{N} \sum_{i=1}^N \delta_i \cdot w_i, \quad \text{where } \delta_i = \begin{cases} +1, & \text{if } \hat{y}_i = \text{positive} \\ -1, & \text{if } \hat{y}_i = \text{negative} \end{cases} \quad (6)$$

Here, w_i is the weight of the i -th comment, $\hat{y}_i$ is the predicted sentiment label, and N is the total number of comments for the game.

Table 3: Top Games by Comment Count in the Test Set

Game Name	Comment Count
Grand Theft Auto V	1551
Tom Clancy’s Rainbow Six® Siege	812
Counter-Strike 2	780
Dead by Daylight	645
Call of Duty®: Black Ops III	585
Sea of Thieves: 2024 Edition	555
ELDEN RING	536
Total War: WARHAMMER III	486
Warframe	446
Call of Duty®	445
Apex Legends™	443
Noita	403
Wallpaper Engine	317
Dragon’s Dogma 2	224
NARAKA: BLADEPOINT	163

3.4.2 Real-time Steam Ranking

We ranked the 15 games listed above based on their user approval ratings as shown on their respective Steam store pages[3], as presented in Table 4. Note that these statistics were collected as of June 13, 2025, 16:00 (UTC+8), and the ranking reflects only the data available up to that time.

Table 4: Game Rankings by Steam User Approval Rate (as of June 13, 2025, 16:00 (UTC+8))

Rank	Game Name	Approval Rate (%)
1	Wallpaper Engine	98.034
2	Noita	95.258
3	ELDEN RING	92.895
4	Sea of Thieves: 2024 Edition	89.248
5	Warframe	87.053
6	Counter-Strike 2	86.678
7	Call of Duty®: Black Ops III	84.335
8	Tom Clancy’s Rainbow Six® Siege	83.645
9	Dead by Daylight	79.065
10	Total War: WARHAMMER III	72.765
11	NARAKA: BLADEPOINT	69.506
12	Grand Theft Auto V	67.273
13	Apex Legends™	67.143
14	Dragon’s Dogma 2	60.908
15	Call of Duty®	58.756

3.4.3 Ranking on Enhanced Naive Bayes Model results

The final score for each game based on Enhanced Naive Bayes Model are shown in Table 5.

Compared with real-time Steam ranking, both rankings demonstrate a high degree of consistency at the top of the list. In particular, *Wallpaper Engine* and *Noita* are ranked first and second in both models, indicating that the classifier successfully captures strong positive sentiment when user consensus is high. Likewise, *ELDEN RING* and *Counter-Strike 2* consistently appear among the top five, reflecting their overall positive reception on the platform.

However, discrepancies emerge in the mid-to-lower sections of the rankings. For example, *Total War: WARHAMMER III*, which holds a relatively moderate approval rate (72.765%), is ranked last in our model, suggesting that the classifier may have captured a concentration of negative sentiment in a small subset of recent reviews. Conversely, *Call of Duty®: Black Ops III* is ranked third by the model but only seventh in Steam approval, potentially due to topical variations in the training set.

These differences highlight that while the model aligns well with global user trends for highly polarized titles, it may underperform when sentiment is more nuanced or temporally variable. Future improvements may involve training type-specific Naive Bayes models tailored to individual game genres, allowing the classifier to better capture domain-specific sentiment patterns and vocabulary usage.

Table 5: Game Rankings on Enhanced Naive Bayes Model

Rank	Game Name	Score
1	Wallpaper Engine	0.11451
2	Noita	0.03616
3	Call of Duty®: Black Ops III	0.02307
4	ELDEN RING	0.02252
5	Counter-Strike 2	0.02044
6	Sea of Thieves: 2024 Edition	0.01907
7	Tom Clancy’s Rainbow Six® Siege	0.01851
8	Apex Legends™	0.00332
9	Warframe	-0.00604
10	Dead by Daylight	-0.03654
11	NARAKA: BLADEPOINT	-0.04911
12	Call of Duty®	-0.05312
13	Grand Theft Auto V	-0.07001
14	Dragon’s Dogma 2	-0.08050
15	Total War: WARHAMMER III	-0.10697

3.4.4 Ranking on Hidden Markov Model Results

Table 6 presents the 15 games ranked by their total weighted sentiment scores as produced by the HMM classifier, sorted in descending order.

Table 6: Game Rankings on HMM Classifier

Rank	Game Name	Score
1	Wallpaper Engine	0.15272
2	ELDEN RING	0.15021
3	Tom Clancy’s Rainbow Six® Siege	0.14376
4	Counter-Strike 2	0.13803
5	Warframe	0.13213
6	Noita	0.12000
7	Apex Legends™	0.10927
8	Dead by Daylight	0.10672
9	Sea of Thieves: 2024 Edition	0.10300
10	Call of Duty®: Black Ops III	0.09370
11	Call of Duty®	0.06316
12	NARAKA: BLADEPOINT	0.04291
13	Grand Theft Auto V	0.01102
14	Total War: WARHAMMER III	-0.01189
15	Dragon’s Dogma 2	-0.01459

The HMM ranking places *Wallpaper Engine* and *ELDEN RING* at the very top, reflecting their overwhelmingly positive sequential sentiment patterns. They are followed by *Tom Clancy’s Rainbow Six Siege* and *Counter-Strike 2*, which also exhibit strong positive comment sequences.

Mid-ranked titles such as *Warframe*, *Noita*, and *Apex Legends* maintain high scores (above 0.10), indicating consistent positive feedback across users’ comment sequences. *Dead by Daylight* and *Sea of Thieves* likewise appear in the top ten, underscoring the HMM’s capacity to capture sustained positive sentiment over longer reviews.

Lower down, *Call of Duty: Black Ops III* and *Call of Duty* register moderate positive scores but start to show more mixed comment sequences. *NARAKA: BLADEPOINT* and *Grand Theft Auto V* fall near zero, indicating roughly balanced positive and negative weights. Finally, *Total War: WARHAMMER III* and *Dragon’s Dogma 2* receive negative scores, suggesting a predominance of negatively weighted sequences in their comments.

Overall, the HMM-based ranking closely matches the top titles identified by the Enhanced Naive Bayes model but diverges in the mid-range, where sequential patterns (e.g. recent negative streaks)

have greater influence. A promising direction for future work is to fuse the HMM sentiment scores with the Naive Bayes scores to mitigate such temporal fluctuations and achieve a more stable ranking.

References

- [1] Narayanan, V., Arora, I., & Bhatia, A. Fast and accurate sentiment classification using an enhanced Naive Bayes model. Department of Electronics Engineering, Indian Institute of Technology (BHU), Varanasi, India, 2011.
- [2] Zhao, X., & Ohsawa, Y. Sentiment analysis on the online reviews based on hidden Markov model. *Journal of Advances in Information Technology*, 2018. Available at: <https://www.jait.us>
- [3] Steam Community. User Reviews. Available at: <https://store.steampowered.com/> Accessed at: June 13, 2025, 16:00 (UTC+8)